National University of Computer and Emerging Sciences

Islamabad Campus

Data Structures (CS2001)

Sessional-I Exam

Course Instructor(s):
Dr. Hina Ayaz, Ms. Noorul Ain
Section(s): (CS A&B)

Total Time (Hrs): 1
Total Marks: 60
Total Questions: 6

Date: Feb 24, 2026

Roll No _____ **Course Section** _____ **Student Signature** _____

Do not write below this line.

Attempt all the questions.

DO NOT OPEN THE QUESTION BOOK OR START UNTIL INSTRUCTED.

Instructions:

1. Attempt on question paper. Attempt all of them. Read the question carefully, understand the question, and then attempt it.
2. No additional sheet will be provided for rough work. Use the back of the last page for rough work
3. If you need more space write on the back side of the paper and clearly mark question and part number etc.
4. Calculator sharing is strictly prohibited.
5. Use permanent ink pens only. Any part done using soft pencil will not be marked and cannot be claimed for rechecking

[CLO 1: Demonstrate basic concepts of data structure and algorithms]

Q1: Do as directed [3+7=10 marks]

a. Given the array:

```
int arr[] = {10, 20, 30, 40};
```

and assuming `list` points to the first element of the array, determine the output of the statement below and explain the reason for your answer.

```
cout << (*list + 3) << endl;
```

Output:

b. Perform a dry run to produce the given output using a 2D array (array of arrays). The elements of the 2D array are:

1 2 3 4

5 6 7 8

9 10 11 12

[CLO 2: Evaluate different data structures in terms of memory complexity and time]

Q2: For the given code calculate the running time of each instruction separately and then calculate the complete time complexity in the given table. [10 marks]

```
#include <iostream>
using namespace std;

int main() {
   const int MAX = 10;
   long NIC[MAX] = {1111, 2222, 3333};
   string Name[MAX] = {"Ali", "Ahmed",
"Sara"};
   string Address[MAX] = {"Lahore",
"Karachi", "Islamabad"};
   int count = 3;
   long searchNIC;
   bool found = false;
   cout << "Enter NIC: ";
   cin >> searchNIC;
   int i = 0;
   while (i < count) {
      if (NIC[i] == searchNIC) {
         found = true;
         break;
      }
```

```
    i++;
  }

  if (found) {
     cout << "Contract already exists. Cannot issue new contract.\n";
  } else {
     if (count < MAX) {
        NIC[count] = searchNIC;
        cout << "Enter Name: ";
        cin >> Name[count];
        cout << "Enter Address: ";
        cin >> Address[count];
        count++;
        cout << "Contract issued successfully.\n";
     } else {
        cout << "Database is full. Cannot issue new contract.\n";
     }
  }

  return 0;
}
```

Step No	Statement	Frequency (Worst Case)	Time per Execution	Total Time
1	`int i = 0;`			
2	`while (i < count)` (comparison)			
3	`if (NIC[i] == searchNIC)`			
4	`found = true;`			
5	`break;`			
6	`i++;`			
7	`if (found)`			
8	Insertion statements (if not found)			

Q3. Calculate the Big-O complexity for the following code snippets and write them in the given boxes [3+3+4=10 marks]

a.

```
void SN(int n) {
  while (n > 1) {
    n = n / 2;
    cout << n << " ";
  }
}
```

b.

```
void exam(int n) {
  for (int i = 1; i < n; i *= 2) {
    cout << i << " ";
```

```
    }
}
```

c.

```
int main() {
    int n = 1024;
    for (int i = n; i > 1; i /= 2) {
        for (int j = n; j > 1; j /= 2) {
            for (int k = n; k > 1; k /= 2) {
                cout << "*";  } } }
    return 0;}
```

Q4. You are given an array of size n. **[Total Marks=20]**

Case A: Array is almost sorted except last element is smallest.

Case B: Array is completely reverse sorted.

Tasks:

1. Rank the following algorithms from best to worst for Case A: **[Marks =1.5]**
   a. Insertion sort
   b. Bubble sort (optimized)
   c. Selection sort

2. Rank them for Case B. **[Marks=1.5]**

3. Which algorithm has: **[Marks =3]**
   a. Minimum swaps? _____
   b. Minimum comparisons? _____
   c. Best practical performance? _____

4. If array size is 100,000: **[Marks =2]**
   a. Which algorithm becomes impractical? _____
   b. Why? _____

5. Suppose you must search 500 times after sorting once. **[Marks =2]**
   Should you:

National University of Computer and Emerging Sciences

Islamabad Campus

a. Use linear search each time?

b. Sort once + use binary search? Justify with complexity analysis.

Q5.

You are given an integer array nums of length n where:

- Each element is in [1, n]
- Each number appears **at most twice**

Complete the function findDuplicates using **O(n) time**, and **O(1) extra space** (excluding output).

[Marks =5]

```
#include <iostream>
using namespace std;

int* findDuplicates(int nums[], int n, int &outSize) {
   static int result[100]; // assume max duplicates <= 100
   outSize = 0;
   for(int i = 0; i < n; i++) {
      int val = nums[i];

      ____________________________

      int index = ____________________________ ;


      if( ____________________________ ) {
         ____________________________ ;
      } else {
         ____________________________ ;
      }
   }

   return result;
}

int main() {
   int nums[] = {4,3,2,7,8,2,3,1};
   int n = 8;
   int outSize;
   int* res = findDuplicates(nums, n, outSize);

   for(int i = 0; i < outSize; i++)
      cout << res[i] << " ";
```

```
    cout << endl;

    return 0;
}
```

Q6. [Marks =5]

You are given an array of distinct integers called candidates and a target integer target.
Complete the following program to find all unique combinations of numbers from candidates that sum up exactly to target. You can choose the same number multiple times in a combination. Two combinations are considered different if the frequency of at least one chosen number is different. You may return or print the combinations in any order. The input is such that the total number of unique combinations is guaranteed to be small enough (e.g., less than 150).

Example 1:	**Example 2:**
Input: candidates = [2,3,6,7], target = 7 Output: [2,2,3] [7]	Input: candidates = [2,3,5], target = 8 Output: [2,2,2,2] [2,3,3] [3,5]

Fill in the missing code:

```
#include <iostream>
using namespace std;
#define MAX_COMB 150
#define MAX_LEN 50
int comb[MAX_LEN];
int combCount = 0;

void combinationSumHelper(int candidates[], int n, int target, int index, int len) {
    if (target == _________) {
        for (int i = 0; i < len; i++)
            cout << comb[i] << " ";
        cout << endl;
        ________________
        return;
    }

    for (int _______________; i++) {
        if (_________________) {
            comb[len] = _________________
            combinationSumHelper(candidates, n, target - candidates[i], i, len + 1);
        }
    }
}

int main() {
    int candidates[5] = {2, 3, 4, 5, 7};
    int n = 5;
```